Documentation Technique

Application Web du Centre Equestre de Pontchateau

Architecture, base de donnees, logique metier et deploiement

Equitation Harmonique -- Loire-Atlantique, France

Contexte : Projet BTS SIO option SLAM

Version du document : 1.0

Date : Mars 2026

Framework : Laravel 12 -- PHP 8.4

Base de donnees : MariaDB 11

Table des matieres

1. Presentation du projet
2. Stack technique
3. Architecture applicative
4. Installation et mise en route
5. Structure des fichiers du projet
6. Base de donnees : schema et migrations
7. Modeles Eloquent et relations
8. Routage de l'application
9. Controleurs : logique de traitement
10. Validation des donnees
11. Authentification et securite
12. Gestion des roles et permissions
13. Logique metier : systeme d'abonnement
14. Vues et templating Blade
15. Frontend : CSS, JavaScript et build
16. Seeders et factories
17. Tests et qualite de code
18. Environnement Docker et deploiement
19. Configuration de l'environnement
20. Maintenance et commandes utiles
21. Axes d'evolution

1. Presentation du projet

1.1 Contexte

Cette application a ete developpee dans le cadre du BTS SIO option SLAM. Elle repond au besoin du Centre Equestre de Pontchateau (Loire-Atlantique) de disposer d'une plateforme web pour :

- Presenter le centre, sa philosophie et ses installations (site vitrine)
- Permettre aux adherents de consulter le planning et de s'inscrire aux cours en ligne
- Gerer les abonnements selon deux formules (annuelle et carte a credits)
- Presenter la cavalerie, les galeries photos et la revue de presse

1.2 Perimetre fonctionnel

Module	Description
Site vitrine	9 pages statiques de presentation
Authentification	Inscription, connexion, verification email, reinitialisation mot de passe
Planning	Calendrier interactif avec FullCalendar, gestion des recurrences
Inscriptions	Systeme d'inscription aux cours avec gestion de credits
Stages	Consultation des stages avec programme journalier et tarifs
Cavalerie	Presentation des chevaux/poneys avec page hommage
Galeries	Albums photos avec recherche
Revue de presse	Articles avec liens externes
Tarifs	Grille tarifaire dynamique par categorie et section
Profil	Gestion du compte utilisateur

1.3 Architecture logicielle

L'application suit le pattern **MVC (Model-View-Controller)** impose par le framework Laravel. Elle est de type **monolithique** avec rendu cote serveur via le moteur de templates Blade.

2. Stack technique

2.1 Backend

Composant	Version	Role
PHP	8.4	Langage serveur
Laravel	12.x	Framework MVC
MariaDB	11	Systeme de gestion de base de donnees relationnelle
Redis	Alpine	Cache, sessions et files d'attente
Composer	2.x	Gestionnaire de dependances PHP

2.2 Frontend

Composant	Version	Role
Bootstrap	5.3.8	Framework CSS (composants, grille, responsive)
Tailwind CSS	3.1.0	Framework CSS utilitaire (complementaire)
Alpine.js	3.4.2	Framework JavaScript reactif leger
Vite	7.0.7	Outil de build et serveur de developpement
FullCalendar	6.1.10	Composant calendrier interactif (charge par CDN)
Axios	1.11.0	Client HTTP pour requetes asynchrones
Font Awesome	CDN	Bibliotheque d'icones
Popper.js	2.11.8	Positionnement des tooltips et popovers Bootstrap

2.3 Polices de caracteres

Police	Type	Usage
Cinzel	Serif	Titres et en-tetes
Lato	Sans-serif	Corps de texte

2.4 Palette de couleurs

Designation	Code hexadecimal	Usage
Primaire (brun dore)	`#bf9b6e`	Boutons, accents, liens, bordures
Secondaire (rouge fonce)	`#340604`	En-tetes, titres, arriere-plans fonces
Fond principal	`#fcf9f2`	Arriere-plan des pages

2.5 Dependances PHP (composer.json)

Dependances de production

Package	Version	Role
`php`	^8.2	Version minimum de PHP
`laravel/framework`	^12.0	Framework principal
`laravel/tinker`	^2.10.1	Console interactive (REPL)
`silber/bouncer`	^1.0	Gestion des roles et permissions

Dependances de developpement

Package	Version	Role
`fakerphp/faker`	^1.23	Generation de donnees factices
`larastan/larastan`	^3.8	Extension PHPStan pour Laravel
`laravel/breeze`	^2.3	Scaffolding d'authentification
`laravel/pail`	^1.2.2	Visualisation des logs en temps reel
`laravel/pint`	^1.24	Formatage de code PHP (PSR-12)
`laravel/sail`	^1.41	Environnement Docker local
`mockery/mockery`	^1.6	Framework de mocking pour tests
`nunomaduro/collision`	^8.6	Affichage ameliore des erreurs en CLI
`pestphp/pest`	^4.1	Framework de tests (surcouche PHPUnit)
`pestphp/pest-plugin-laravel`	^4.0	Plugin Pest pour Laravel
`phpstan/phpstan`	^2.1	Analyse statique du code PHP

2.6 Dependances npm (package.json)

Dependances de production

Package	Version	Role
`@popperjs/core`	^2.11.8	Positionnement elements UI (Bootstrap)
`bootstrap`	^5.3.8	Framework CSS

Dependances de developpement

Package	Version	Role
`@tailwindcss/forms`	^0.5.2	Plugin Tailwind pour formulaires
`@tailwindcss/vite`	^4.0.0	Integration Tailwind avec Vite
`alpinejs`	^3.4.2	Framework JS reactif
`autoprefixer`	^10.4.2	Prefixes CSS navigateurs

`axios`	^1.11.0	Client HTTP
`concurrently`	^9.0.1	Execution parallele de commandes
`laravel-vite-plugin`	^2.0.0	Integration Laravel/Vite
`postcss`	^8.4.31	Transformation CSS
`tailwindcss`	^3.1.0	Framework CSS utilitaire
`vite`	^7.0.7	Build tool

3. Architecture applicative

3.1 Pattern MVC

```
Requete HTTP du navigateur
        |
        v
   Routes (web.php / auth.php)
        |
        v
   Middleware (auth, guest, verified, throttle, signed)
        |
        v
   Controller
        |
        v
   Model (Eloquent ORM) <---> MariaDB
        |
        v
   Vue (Blade Template)
        |
        v
   Reponse HTTP vers le navigateur
```

3.2 Couches de l'application

Couche	Repertoire	Responsabilite
Routes	`routes/`	Definition des endpoints HTTP et association aux controleurs
Middleware	`app/Http/Middleware/`	Filtrage et traitement des requetes avant le controleur
Controleurs	`app/Http/Controllers/`	Logique de traitement des requetes
Form Requests	`app/Http/Requests/`	Validation des donnees entrantes
Modeles	`app/Models/`	Representation des entites et acces aux donnees (ORM)
Vues	`resources/views/`	Templates Blade pour le rendu HTML
Composants	`app/View/Components/`	Composants de mise en page reutilisables
Providers	`app/Providers/`	Configuration et enregistrement des services
Migrations	`database/migrations/`	Definition et evolution du schema de base de donnees
Seeders	`database/seeders/`	Peuplement initial de la base de donnees
Factories	`database/factories/`	Generation de donnees factices pour les tests

3.3 Point d'entree de l'application

Le fichier `bootstrap/app.php` configure l'application Laravel :

```php
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
```

```
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware): void {
        //
    })
    ->withExceptions(function (Exceptions $exceptions): void {
        //
    })->create();
```

Le fichier `bootstrap/providers.php` declare le seul service provider :

```
return [
    App\Providers\AppServiceProvider::class,
];
```

4. Installation et mise en route

4.1 Pre-requis

- Docker et Docker Compose (pour l'environnement Sail)
- Git
- Node.js 18+ et npm
- Composer 2.x (pour l'installation initiale)

4.2 Installation automatisee

```
git clone <url-du-depot>
cd E6_app_l-gere
composer setup
```

La commande `composer setup` execute sequentiellement :

```
"setup": [
    "composer install",
    "@php -r \"file_exists('.env') || copy('.env.example', '.env');\"",
    "@php artisan key:generate",
    "@php artisan migrate --force",
    "npm install",
    "npm run build"
]
```

4.3 Demarrage en developpement

```
composer dev
```

Cette commande utilise `concurrently` pour lancer simultanement :

Processus	Commande	Port
Serveur Laravel	`php artisan serve`	8000
Worker de queue	`php artisan queue:listen --tries=1`	--
Serveur Vite (hot reload)	`npm run dev`	5173

4.4 Demarrage avec Docker (Sail)

```
./vendor/bin/sail up -d
./vendor/bin/sail artisan migrate
./vendor/bin/sail artisan db:seed
./vendor/bin/sail npm run build
```

4.5 Peuplement de la base de donnees

```
php artisan db:seed
```

Cette commande cree les donnees de demonstration via le `DatabaseSeeder` (cf. section 16).

5. Structure des fichiers du projet

```
E6_app_l-gere/
|-- app/
|   |-- Http/
|   |   |-- Controllers/
|   |   |   |-- ActualiteController.php
|   |   |   |-- Auth/
|   |   |   |   |-- AuthenticatedSessionController.php
|   |   |   |   |-- ConfirmablePasswordController.php
|   |   |   |   |-- EmailVerificationNotificationController.php
|   |   |   |   |-- EmailVerificationPromptController.php
|   |   |   |   |-- NewPasswordController.php
|   |   |   |   |-- PasswordController.php
|   |   |   |   |-- PasswordResetLinkController.php
|   |   |   |   |-- RegisteredUserController.php
|   |   |   |   |-- VerifyEmailController.php
|   |   |   |-- GalleryController.php
|   |   |   |-- PlanningController.php
|   |   |   |-- PressReviewController.php
|   |   |   |-- ProfileController.php
|   |   |   |-- StageController.php
|   |   |   |-- TarifController.php
|   |   |-- Middleware/
|   |   |   |-- ScopeBouncer.php
|   |   |-- Requests/
|   |       |-- ProfileUpdateRequest.php
|   |-- Models/
|   |   |-- Adherent.php
|   |   |-- Event.php
|   |   |-- EventPrice.php
|   |   |-- EventType.php
|   |   |-- Gallery.php
|   |   |-- GalleryPhoto.php
|   |   |-- Horse.php
|   |   |-- PressReview.php
|   |   |-- Tarif.php
|   |   |-- Tariff.php (vide, non utilise)
|   |   |-- User.php
|   |-- Providers/
|   |   |-- AppServiceProvider.php
|   |-- View/
|       |-- Components/
|           |-- AppLayout.php
|           |-- GuestLayout.php
|-- bootstrap/
|   |-- app.php
|   |-- providers.php
|-- config/
|   |-- app.php, auth.php, cache.php, database.php,
|   |-- filesystems.php, logging.php, mail.php,
|   |-- queue.php, services.php, session.php
|-- database/
|   |-- factories/ (7 fichiers)
|   |-- migrations/ (19 fichiers)
|   |-- seeders/ (8 fichiers)
|-- public/
|   |-- index.php
|   |-- css/style.css
|   |-- img/, img-stock/, img-style/
|-- resources/
|   |-- css/app.css
|   |-- js/app.js
```

```
|   |-- views/ (templates Blade)
|-- routes/
|   |-- web.php
|   |-- auth.php
|   |-- console.php
|-- storage/
|-- tests/
|   |-- Feature/, Unit/
|   |-- Pest.php, TestCase.php
|-- compose.yaml
|-- composer.json
|-- package.json
|-- phpstan.neon
|-- phpunit.xml
|-- vite.config.js
|-- tailwind.config.js
|-- postcss.config.js
```

6. Base de donnees : schema et migrations

6.1 Schema relationnel

```
users ──────────── 1:1 ──────────── adherents
  |
  | N:M (event_user)
  |   pivot: date, is_cancelled,
  |          horse_id
  v
events ──────────── N:M ──────────── event_types
  |               (events_asigne_type)
  |
  |── 1:N ──────── event_prices

galleries ──────── 1:N ──────────── gallery_photos

horses ────────── (lie via event_user.horse_id)

horse_schedules ── FK: horse_id, user_id

press_reviews ──── (independant)

tarifs ──────────── (independant)

Systeme Bouncer : abilities, roles, assigned_roles, permissions
```

6.2 Liste complete des migrations

N.	Fichier de migration	Table(s)	Operation
1	`0001_01_01_000000_create_users_table`	`users` , `password_reset_tokens` , `sessions`	Creation
2	`0001_01_01_000001_create_cache_table`	`cache` , `cache_locks`	Creation
3	`0001_01_01_000002_create_jobs_table`	`jobs` , `job_batches` , `failed_jobs`	Creation
4	`2025_12_03_125355_create_events_table`	`events`	Creation
5	`2025_12_03_130707_create_event_user_table`	`event_user`	Creation
6	`2025_12_03_131235_add_role_to_users_table`	`users`	Ajout colonne `role`
7	`2025_12_03_153109_create_event_types_table`	`event_types`	Creation
8	`2025_12_03_153146_create_event_event_type_table`	`events_asigne_type`	Creation
9	`2025_12_03_154547_create_bouncer_tables`	`abilities` , `roles` , `assigned_roles` , `permissions`	Creation
10	`2025_12_05_104822_add_phone_number_to_users`	`users`	Ajout colonne `phone_number`
11	`2025_12_05_113035_create_horses_table`	`horses`	Creation
12	`2025_12_10_070948_create_adherents_table`	`adherents`	Creation

13	`2025_12_10_072341_add_date_to_event_user`	`event_user`	Migration vide
14	`2025_12_10_072351_add_date_to_event_user`	`event_user`	Ajout colonne `date`
15	`2025_12_10_074406_add_is_cancelled_to_event_user`	`event_user`	Ajout colonne `is_cancelled`
16	`2026_01_12_092357_create_press_reviews_table`	`press_reviews`	Creation
17	`2026_01_12_130147_create_galleries_table`	`galleries`	Creation
18	`2026_01_12_130147_create_gallery_photos_table`	`gallery_photos`	Creation
19	`2026_01_12_165536_create_tarifs_table`	`tarifs`	Creation
20	`2026_01_13_082830_add_daily_schedule_to_events`	`events`	Ajout colonne `daily_schedule` (JSON)
21	`2026_01_19_084801_add_horse_id_to_event_user`	`event_user`	Ajout FK `horse_id`
22	`2026_01_22_083046_create_horse_schedules_table`	`horse_schedules`	Creation
23	`2026_01_29_082655_create_event_prices_table`	`event_prices`	Creation
24	`2026_02_13_102302_add_is_deceased_to_horses`	`horses`	Ajout colonne `is_deceased`

6.3 Detail des tables

Table `users`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`name`	varchar(255)	NOT NULL	Nom complet
`email`	varchar(255)	NOT NULL, UNIQUE	Adresse email (identifiant de connexion)
`email_verified_at`	timestamp	NULLABLE	Date de verification de l'email
`password`	varchar(255)	NOT NULL	Mot de passe hashe (bcrypt)
`phone_number`	varchar(255)	NOT NULL	Numero de telephone
`role`	varchar(255)	DEFAULT 'user'	Role (admin, enseignant, user)
`remember_token`	varchar(100)	NULLABLE	Jeton "Se souvenir de moi"
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `adherents`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`user_id`	bigint unsigned	FK -> users.id, ON DELETE CASCADE	Reference utilisateur
`formule`	varchar(255)	NULLABLE	Type d'abonnement : `annee`, `carte` ou null
`value`	integer	DEFAULT 0	Nombre de credits restants (formule carte)
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `events`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`name`	varchar(255)	NOT NULL	Nom de l'evenement
`description`	text	NOT NULL	Description detaillee
`start_date`	datetime	NOT NULL	Date et heure de debut
`end_date`	datetime	NOT NULL	Date et heure de fin
`daily_schedule`	json	NULLABLE	Planning journalier pour les stages
`created_at`	timestamp	NULLABLE	Date de creation

`updated_at`	timestamp	NULLABLE	Date de modification

Table `event_user` (table pivot)

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`user_id`	bigint unsigned	FK -> users.id, ON DELETE CASCADE	Reference utilisateur
`event_id`	bigint unsigned	FK -> events.id, ON DELETE CASCADE	Reference evenement
`date`	date	NULLABLE	Date de la seance (null = inscription a l'annee)
`is_cancelled`	boolean	DEFAULT false	Inscription annulee
`horse_id`	bigint unsigned	FK -> horses.id, NULLABLE, ON DELETE SET NULL	Cheval attribue
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `event_types`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`name`	varchar(255)	NOT NULL	Nom du type (cours, stage, concours)
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `events_asigne_type` (table pivot)

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`event_id`	bigint unsigned	FK -> events.id, ON DELETE CASCADE	Reference evenement
`event_type_id`	bigint unsigned	FK -> event_types.id, ON DELETE CASCADE	Reference type
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `event_prices`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique

`event_id`	bigint unsigned	FK -> events.id, ON DELETE CASCADE	Reference evenement
`label`	varchar(255)	NOT NULL	Intitule du tarif (ex : "Stage complet")
`price`	decimal(10,2)	NOT NULL	Prix en euros
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `horses`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`name`	varchar(255)	NOT NULL	Nom de l'animal
`birth_date`	date	NOT NULL	Date de naissance
`photo_path`	varchar(255)	NULLABLE	Chemin de la photo
`type`	enum('cheval','poney')	NOT NULL	Type d'animal
`is_deceased`	boolean	DEFAULT false	Animal decede
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `horse_schedules`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`horse_id`	bigint unsigned	FK -> horses.id, ON DELETE CASCADE	Reference cheval
`user_id`	bigint unsigned	FK -> users.id, NULLABLE, ON DELETE SET NULL	Reference utilisateur
`activity`	varchar(255)	NOT NULL	Description de l'activite
`start_time`	datetime	NOT NULL	Debut de l'activite
`end_time`	datetime	NOT NULL	Fin de l'activite
`comments`	text	NULLABLE	Commentaires
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `galleries`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`title`	varchar(255)	NOT NULL	Titre de la galerie
`date`	date	NOT NULL	Date de la galerie

`cover_image`	varchar(255)	NULLABLE	Image de couverture
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `gallery_photos`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`gallery_id`	bigint unsigned	FK -> galleries.id, ON DELETE CASCADE	Reference galerie
`image_path`	varchar(255)	NOT NULL	Chemin de l'image
`title`	varchar(255)	NULLABLE	Titre de la photo
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `press_reviews`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`title`	varchar(255)	NOT NULL	Titre de l'article
`description`	text	NOT NULL	Description ou resume
`date`	date	NOT NULL	Date de publication
`link`	varchar(255)	NOT NULL	URL de l'article
`created_at`	timestamp	NULLABLE	Date de creation
`updated_at`	timestamp	NULLABLE	Date de modification

Table `tarifs`

Colonne	Type	Contrainte	Description
`id`	bigint unsigned	PK, auto-increment	Identifiant unique
`category`	enum('cheval','poney')	NOT NULL	Categorie
`section`	varchar(255)	NOT NULL	Section (enseignement, option, cartes, a_la_carte, proprietaire)
`title`	varchar(255)	NOT NULL	Intitule du tarif
`description`	varchar(255)	NULLABLE	Description complementaire
`price`	decimal(8,2)	NOT NULL	Prix en euros
`promo_text`	varchar(255)	NULLABLE	Texte promotionnel
`frequency`	varchar(255)	NULLABLE	Frequence de paiement
`created_at`	timestamp	NULLABLE	Date de creation

`updated_at`	timestamp	NULLABLE	Date de modification

Tables Bouncer (systeme de roles)

Table	Colonnes principales	Description
`abilities`	id, name, title, entity_id, entity_type, only_owned, options, scope	Permissions definies
`roles`	id, name, title, scope	Roles (admin, enseignant)
`assigned_roles`	id, role_id, entity_id, entity_type, restricted_to_id, restricted_to_type, scope	Attribution roles-entites
`permissions`	id, ability_id, entity_id, entity_type, forbidden, scope	Attribution permissions-entites

7. Modeles Eloquent et relations

7.1 Diagramme des relations

```
User ─── hasOne ──> Adherent
User ─── belongsToMany ──> Event
     (pivot event_user : date, is_cancelled, horse_id)

Event ─── belongsToMany ──> EventType
      (pivot events_asigne_type)
Event ─── hasMany ──> EventPrice
Event ─── belongsToMany ──> User

Gallery ─── hasMany ──> GalleryPhoto

Horse (pas de relation Eloquent definie,
       lie via event_user.horse_id)
PressReview (independant)
Tarif (independant)
```

7.2 Modele User

Fichier : `app/Models/User.php`

Traits et interfaces :

- `HasFactory` : support des factories pour les tests
- `Notifiable` : support des notifications Laravel
- `HasRolesAndAbilities` : trait Bouncer pour les roles et permissions
- Implemente `MustVerifyEmail` : verification obligatoire de l'email

Attributs fillable : `name`, `email`, `password`, `role`, `phone_number`

Attributs hidden : `password`, `remember_token`

Casts :

- `email_verified_at` -> `datetime`
- `password` -> `hashed` (hashage automatique via le cast Laravel)

Relations :

Methode	Type	Modele lie	Details
`adherent()`	HasOne	Adherent	Profil d'abonnement
`events()`	BelongsToMany	Event	Pivot : date, is_cancelled, horse_id

Evenement de modele (`booted()`) : a la creation d'un utilisateur, un enregistrement `Adherent` est automatiquement cree avec `formule = null` et `value = 0`.

```php
protected static function booted(): void
{
    static::created(function (User $user) {
        $user->adherent()->create([
```

```
            'formule' => null,
            'value' => 0,
        ]);
    });
}
```

7.3 Modele Adherent

Fichier : `app/Models/Adherent.php`

Attributs fillable : `user_id` , `formule` , `value`

Relations :

Methode	Type	Modele lie
`user()`	BelongsTo	User

7.4 Modele Event

Fichier : `app/Models/Event.php`

Attributs fillable : `name` , `description` , `start_date` , `end_date` , `daily_schedule`

Casts :

- `start_date` -> `datetime`
- `end_date` -> `datetime`
- `daily_schedule` -> `array` (JSON decode automatique)

Relations :

Methode	Type	Modele lie	Table pivot
`users()`	BelongsToMany	User	event_user (date, is_cancelled, horse_id)
`eventTypes()`	BelongsToMany	EventType	events_asigne_type
`prices()`	HasMany	EventPrice	--

7.5 Modele EventType

Fichier : `app/Models/EventType.php`

Attributs fillable : `name`

Relations :

Methode	Type	Modele lie	Table pivot
`events()`	BelongsToMany	Event	events_asigne_type

7.6 Modele EventPrice

Fichier : `app/Models/EventPrice.php`

Attributs fillable : `event_id` , `label` , `price`

Relations :

Methode	Type	Modele lie
`event()`	BelongsTo	Event

7.7 Modele Horse

Fichier : `app/Models/Horse.php`

Attributs fillable : `name` , `birth_date` , `photo_path` , `type` , `is_deceased`

Casts :

- `birth_date` -> `date`
- `is_deceased` -> `boolean`

Aucune relation Eloquent definie. Le lien avec les inscriptions se fait via la colonne `horse_id` de la table pivot `event_user` .

7.8 Modele Gallery

Fichier : `app/Models/Gallery.php`

Attributs fillable : `title` , `date` , `cover_image`

Casts : `date` -> `date`

Relations :

Methode	Type	Modele lie
`photos()`	HasMany	GalleryPhoto

7.9 Modele GalleryPhoto

Fichier : `app/Models/GalleryPhoto.php`

Attributs fillable : `gallery_id` , `image_path` , `title`

Relations :

Methode	Type	Modele lie
`gallery()`	BelongsTo	Gallery

7.10 Modele PressReview

Fichier : `app/Models/PressReview.php`

Attributs fillable : `title` , `description` , `date` , `link`

Casts : `date` -> `date`

7.11 Modele Tarif

Fichier : `app/Models/Tarif.php`

Attributs fillable : `category`, `section`, `title`, `description`, `price`, `promo_text`, `frequency`

8. Routage de l'application

8.1 Routes publiques (routes/web.php)

Methode	URI	Controleur / Action	Nom de route
GET	`/`	Closure -> vue `acceuil`	`acceuil`
GET	`/equitation-harmonique`	Closure -> vue `equitation_harmonique`	`equitation-harmonique`
GET	`/projet-pedagogique`	Closure -> vue `projet_pedagogique`	`projet-pedagogique`
GET	`/amenagements`	Closure -> vue `amenagements`	`amenagements`
GET	`/centre-de-loisirs`	Closure -> vue `centre_loisirs`	`centre-de-loisirs`
GET	`/pensions`	Closure -> vue `pension`	`pensions`
GET	`/tarifs`	`TarifController@index`	`tarifs`
GET	`/cavalerie`	Closure (query Horse)	`cavalerie`
GET	`/hommage`	Closure (query Horse)	`hommage`
GET	`/actualites`	`ActualiteController@index`	`actualites`
GET	`/planning`	`PlanningController@index`	`planning`
GET	`/plan-dacces`	Closure -> vue `plan_acces`	`plan-dacces`
GET	`/stages`	`StageController@index`	`stages.index`
GET	`/stages/{stage}`	`StageController@show`	`stages.show`
GET	`/nous-contacter`	Closure -> vue `contact`	`nous-contacter`
GET	`/revue-de-presse`	`PressReviewController@index`	`revue-de-presse`
GET	`/galeries`	`GalleryController@index`	`galeries.index`
GET	`/galeries/{gallery}`	`GalleryController@show`	`galeries.show`

8.2 Routes protegees (middleware `auth`)

Methode	URI	Controleur	Nom de route
GET	`/mon-planning`	`PlanningController@myPlanning`	`my-planning`
POST	`/planning/{event}/subscribe`	`PlanningController@subscribe`	`planning.subscribe`
DELETE	`/planning/{event}/unsubscribe`	`PlanningController@unsubscribe`	`planning.unsubscribe`
GET	`/profile`	`ProfileController@edit`	`profile.edit`
PATCH	`/profile`	`ProfileController@update`	`profile.update`
DELETE	`/profile`	`ProfileController@destroy`	`profile.destroy`

8.3 Routes d'authentification (routes/auth.php)

Middleware `guest` (utilisateurs non connectes) :

Methode	URI	Controleur	Nom
GET	`/register`	`RegisteredUserController@create`	`register`
POST	`/register`	`RegisteredUserController@store`	--
GET	`/login`	`AuthenticatedSessionController@create`	`login`
POST	`/login`	`AuthenticatedSessionController@store`	--
GET	`/forgot-password`	`PasswordResetLinkController@create`	`password.request`
POST	`/forgot-password`	`PasswordResetLinkController@store`	`password.email`
GET	`/reset-password/{token}`	`NewPasswordController@create`	`password.reset`
POST	`/reset-password`	`NewPasswordController@store`	`password.store`

Middleware `auth` (utilisateurs connectes) :

Methode	URI	Controleur	Middleware sup.	Nom
GET	`/verify-email`	`EmailVerificationPromptController`	--	`verification.notice`
GET	`/verify-email/{id}/{hash}`	`VerifyEmailController`	signed, throttle:6,1	`verification.verify`
POST	`/email/verification-notification`	`EmailVerificationNotificationController@store`	throttle:6,1	`verification.send`
GET	`/confirm-password`	`ConfirmablePasswordController@show`	--	`password.confirm`
POST	`/confirm-password`	`ConfirmablePasswordController@store`	--	--
PUT	`/password`	`PasswordController@update`	--	`password.update`
POST	`/logout`	`AuthenticatedSessionController@destroy`	--	`logout`

9. Controleurs : logique de traitement

9.1 PlanningController

Controleur le plus complexe de l'application. Gere l'affichage du calendrier et la logique d'inscription.

Methode `index()` : View

Charge tous les evenements avec leurs types via eager loading (`Event::with('eventTypes')`). Pour chaque evenement, le controleur :

1. Determine le type (cours, stage, concours) en verifiant les types associes
2. Attribue une couleur selon le type
3. Si l'utilisateur est connecte, charge ses inscriptions pour determiner le statut (inscrit globalement ou a des dates specifiques)
4. Formate les donnees pour FullCalendar :
   - **Cours (recurrents)** : utilise `daysOfWeek`, `startTime`, `endTime`, `startRecur` pour un affichage hebdomadaire repetitif
   - **Stages et concours (ponctuels)** : utilise `start` et `end` pour une plage horaire fixe

Methode `subscribe(Request, Event)` : RedirectResponse

Algorithme d'inscription :

```
1. Recuperer l'utilisateur connecte et son profil adherent
2. Recuperer les parametres : date et subscription_type (unique/year)
3. Verifier l'existence du profil adherent (sinon erreur)
4. Determiner le type d'evenement (cours ou stage)
5. Si stage -> refuser et renvoyer message "contacter le centre"
6. Si subscription_type != 'year' :
   a. Calculer la date de l'evenement
   b. Verifier que now() < (date_evenement - 48h)
   c. Si delai insuffisant -> erreur
7. Si formule == 'annee' :
   a. Si type 'year' : verifier pas deja inscrit a l'annee,
      supprimer inscriptions individuelles, creer inscription sans date
   b. Si type 'unique' : verifier pas deja inscrit a cette date,
      creer inscription avec la date
8. Si formule == 'carte' :
   a. Verifier credits > 0
   b. Pour les cours : verifier qu'une date est fournie
   c. Verifier pas deja inscrit a cette date
   d. Creer inscription avec date, decrementer credits
9. Sinon -> erreur "aucune formule active"
```

Methode `myPlanning()` : View

Charge les evenements de l'utilisateur connecte avec les donnees du pivot (date, is_cancelled, horse_id) et les renvoie a la vue `my_planning`.

9.2 StageController

Methode `index()` : View

Requete les evenements ayant le type "stage" (`whereHas('eventTypes', ...)`) dont la date de fin est superieure ou egale a la date du jour, tries par date de debut croissante.

Methode `show(Event $stage)` : View

Verifie que l'evenement est bien de type "stage" (`$stage->eventTypes->contains('name', 'stage')`). Si ce n'est pas le cas, retourne une erreur 404 (`abort(404)`). Sinon, affiche la fiche detaillee.

9.3 TarifController

Methode `index()` : View

Charge tous les tarifs et les regroupe dans un tableau associatif par `category` (cheval/poney) puis par `section` (enseignement, option, cartes, a_la_carte, proprietaire). La section `proprietaire` n'existe que pour la categorie cheval.

9.4 GalleryController

Methode `index()` : View

Charge les galeries triees par date decroissante.

Methode `show(Gallery $gallery)` : View

Charge la galerie et ses photos associees via la relation `photos()`.

9.5 PressReviewController

Methode `index()` : View

Charge les revues de presse triees par date decroissante.

9.6 ActualiteController

Methode `index()` : View

Retourne simplement la vue `actualite` sans donnees dynamiques.

9.7 ProfileController

Methode `edit(Request)` : View

Transmet l'utilisateur courant a la vue `profile.edit`.

Methode `update(ProfileUpdateRequest)` : RedirectResponse

Met a jour les champs valides de l'utilisateur. Si l'email a ete modifie (`isDirty('email')`), reinitialise la date de verification email a null pour forcer une nouvelle verification.

Methode `destroy(Request)` : RedirectResponse

Valide le mot de passe actuel via `validateWithBag('userDeletion', ...)`, deconnecte l'utilisateur, supprime le compte, invalide la session et regenere le jeton CSRF.

10. Validation des donnees

10.1 ProfileUpdateRequest

Fichier : `app/Http/Requests/ProfileUpdateRequest.php`

Etend `FormRequest` de Laravel. Regles de validation :

Champ	Regles
`name`	required, string, max:255
`email`	required, string, lowercase, email, max:255, unique (sauf l'utilisateur courant)

```php
public function rules(): array
{
    return [
        'name' => ['required', 'string', 'max:255'],
        'email' => [
            'required', 'string', 'lowercase', 'email', 'max:255',
            Rule::unique(User::class)->ignore($this->user()->id),
        ],
    ];
}
```

10.2 Validation inline dans PlanningController

Le `PlanningController@subscribe` effectue des validations manuelles :

- Verification de l'existence du profil adherent
- Verification du type d'evenement
- Verification du delai de 48 heures
- Verification des doublons d'inscription
- Verification du solde de credits

Les erreurs sont renvoyees via `back()->with('error', '...')` (messages flash en session).

11. Authentification et securite

11.1 Systeme d'authentification

L'authentification est geree par **Laravel Breeze**, qui fournit :

Fonctionnalite	Controleur	Description
Inscription	`RegisteredUserController`	Creation de compte avec validation
Connexion	`AuthenticatedSessionController`	Authentification email/password
Deconnexion	`AuthenticatedSessionController@destroy`	Invalidation de session
Verification email	`VerifyEmailController`	Lien signe avec expiration
Reinitialisation MDP	`NewPasswordController`	Token unique par email
Confirmation MDP	`ConfirmablePasswordController`	Pour actions sensibles
Mise a jour MDP	`PasswordController`	Depuis le profil utilisateur

11.2 Hashage des mots de passe

Le cast `'password' => 'hashed'` dans le modele `User` assure que le mot de passe est automatiquement hashe via bcrypt avant l'enregistrement en base. En environnement de test, le nombre de rounds bcrypt est reduit a 4 (`BCRYPT_ROUNDS=4`) pour accelerer les tests.

11.3 Protection CSRF

Toutes les requetes POST, PATCH, PUT et DELETE sont protegees par un jeton CSRF genere automatiquement par Laravel. Les formulaires Blade incluent la directive `@csrf`.

11.4 Middleware de securite

Middleware	Role	Routes concernees
`auth`	Verifie que l'utilisateur est connecte	Routes protegees (profil, inscriptions, mon planning)
`guest`	Redirige les utilisateurs connectes	Routes de connexion et d'inscription
`signed`	Verifie la signature cryptographique de l'URL	Verification email
`throttle:6,1`	Limite a 6 requetes par minute	Envoi de verification, verification email
`verified`	Verifie que l'email est confirme	(Disponible mais non utilise dans les routes actuelles)

11.5 Middleware ScopeBouncer

Fichier : `app/Http/Middleware/ScopeBouncer.php`

Middleware prepare pour une future logique de scope multi-tenant avec Bouncer. Actuellement, il transmet la requete sans modification (`return $next($request)`). Son objectif est de scopier les permissions Bouncer a un tenant specifique via `$this->bouncer->scope()->to($tenantId)` .

12. Gestion des roles et permissions

12.1 Package utilise

La gestion des roles et permissions repose sur le package **Silber Bouncer** (`silber/bouncer` v1.x). Il fournit un systeme de roles et d'abilities stockes en base de donnees.

12.2 Integration dans le modele User

Le trait `HasRolesAndAbilities` est ajoute au modele `User` , ce qui lui confere les methodes :

- `$user->isA('admin')` : verification de role
- `$user->can('manage-events')` : verification de permission
- `Bouncer::assign('role')->to($user)` : attribution de role
- `Bouncer::allow('role')->to('ability')` : attribution de permission

12.3 Roles definis

Role	Description	Attribution
`admin`	Administrateur du centre	Via le DatabaseSeeder
`enseignant`	Moniteur d'equitation	Via le DatabaseSeeder
(defaut)	Utilisateur standard	Par defaut (colonne `role` = 'user')

12.4 Permissions (abilities) definies

Permission	Description	Role associe
`manage-events`	Creation, modification et suppression d'evenements	`enseignant`
`view-planning`	Consultation avancee du planning	`enseignant`

12.5 Configuration dans le DatabaseSeeder

```
// Attribution du role admin
Bouncer::assign('admin')->to($admin);

// Attribution du role enseignant avec permissions
Bouncer::assign('enseignant')->to($enseignant);
Bouncer::allow('enseignant')->to(['manage-events', 'view-planning']);
```

13. Logique metier : systeme d'abonnement

13.1 Vue d'ensemble

Le systeme d'abonnement est la logique metier principale de l'application. Il gere l'inscription des cavaliers aux cours via deux formules distinctes.

13.2 Formule Annuelle (`annee`)

```
Utilisateur (adherent.formule = 'annee')
    |
    |-- Inscription annuelle (subscription_type = 'year')
    |   |-- Verifie : pas deja inscrit a l'annee
    |   |-- Supprime les inscriptions individuelles existantes
    |   |-- Cree : event_user avec date = NULL
    |   |-- Resultat : inscrit a toutes les occurrences
    |
    |-- Inscription ponctuelle (subscription_type = 'unique')
        |-- Verifie : delai de 48h respecte
        |-- Verifie : pas deja inscrit a cette date
        |-- Cree : event_user avec date = YYYY-MM-DD
        |-- Aucune deduction de credit
```

13.3 Formule Carte (`carte`)

```
Utilisateur (adherent.formule = 'carte', adherent.value = N)
    |
    |-- Inscription ponctuelle uniquement
        |-- Verifie : adherent.value > 0
        |-- Verifie : delai de 48h respecte
        |-- Verifie : date fournie (pour les cours)
        |-- Verifie : pas deja inscrit a cette date
        |-- Cree : event_user avec date
        |-- Execute : adherent.decrement('value')
        |-- Resultat : adherent.value = N - 1
```

13.4 Regle des 48 heures

Le systeme empeche toute inscription a une seance si la date limite (48 heures avant le cours) est depassee :

```php
$limitDate = $eventDate->copy()->subHours(48);
if (now()->greaterThanOrEqualTo($limitDate)) {
    return back()->with('error',
        'Vous ne pouvez pas vous inscrire moins de 48h avant le debut du cours.');
}
```

Cette regle s'applique uniquement aux inscriptions de type `unique` (pas aux inscriptions annuelles).

13.5 Gestion des types d'evenements dans le calendrier

Type	Couleur	Rendu FullCalendar	Inscription
`cours`	`#bf9b6e`	Recurrent avec `daysOfWeek` + `startTime` / `endTime` + `startRecur`	Seance unique ou annee

`stage`	`#340604`	Ponctuel avec `start` / `end`	Refuse (contacter le centre)
`concours`	`#2c3e50`	Ponctuel avec `start` / `end`	Seance unique

13.6 Creation automatique du profil adherent

Le hook `booted()` du modele `User` cree automatiquement un enregistrement `Adherent` lors de la creation de chaque utilisateur :

```
static::created(function (User $user) {
    $user->adherent()->create([
        'formule' => null,
        'value' => 0,
    ]);
});
```

L'affectation de la formule (`annee` ou `carte`) et des credits est realisee par l'administration.

14. Vues et templating Blade

14.1 Organisation des vues

```
resources/views/
|-- layouts/
|   |-- layout.blade.php          (layout principal)
|   |-- navigation.blade.php      (navigation alternative)
|-- components/                    (composants Blade)
|-- auth/
|   |-- login.blade.php
|   |-- register.blade.php
|   |-- forgot-password.blade.php
|   |-- reset-password.blade.php
|   |-- verify-email.blade.php
|   |-- confirm-password.blade.php
|-- profile/
|   |-- edit.blade.php
|   |-- partials/
|       |-- update-profile-information-form.blade.php
|       |-- update-password-form.blade.php
|       |-- delete-user-form.blade.php
|-- stages/
|   |-- index.blade.php
|   |-- show.blade.php
|-- galeries/
|   |-- index.blade.php
|   |-- show.blade.php
|-- acceuil.blade.php
|-- planning.blade.php
|-- my_planning.blade.php
|-- tarifs.blade.php
|-- cavalerie.blade.php
|-- hommage.blade.php
|-- revue_de_presse.blade.php
|-- equitation_harmonique.blade.php
|-- projet_pedagogique.blade.php
|-- amenagements.blade.php
|-- centre_loisirs.blade.php
|-- pension.blade.php
|-- plan_acces.blade.php
|-- actualite.blade.php
|-- contact.blade.php
|-- welcome.blade.php
```

14.2 Layout principal

Le fichier `resources/views/layouts/layout.blade.php` definit la structure HTML commune a toutes les pages :

- Declaration `<!DOCTYPE html>` et balises `<html lang="fr">`
- Balises `<meta>` SEO : charset, viewport, description, Open Graph
- Chargement des polices Google Fonts (Cinzel, Lato)
- Inclusion de Bootstrap 5 CSS (CDN) et de la feuille de style personnalisee
- Inclusion de Font Awesome (CDN)
- Barre de navigation avec le logo et les liens
- Section `@yield('content')` pour le contenu specifique de chaque page

- Inclusion de Bootstrap 5 JS (CDN)
- Pied de page

14.3 Composants de layout

Composant	Fichier	Role
`AppLayout`	`app/View/Components/AppLayout.php`	Layout pour les utilisateurs connectes
`GuestLayout`	`app/View/Components/GuestLayout.php`	Layout pour les pages d'authentification

14.4 Integration FullCalendar

Le fichier `planning.blade.php` integre FullCalendar 6.1.10 via CDN. La configuration JavaScript :

- **Locale** : francais (`fr`)
- **Vue initiale** : `dayGridMonth` sur desktop, `listMonth` sur mobile
- **Barre d'outils** : navigation (precedent/suivant/aujourd'hui), titre, changement de vue
- **Evenements** : tableau JSON transmis par le controleur via Blade (`@json($events)`)
- **Clic sur evenement** : ouvre un offcanvas Bootstrap avec les details et les boutons d'action

15. Frontend : CSS, JavaScript et build

15.1 Configuration Vite

Fichier : `vite.config.js`

```
import { defineConfig } from 'vite';
import laravel from 'laravel-vite-plugin';

export default defineConfig({
    plugins: [
        laravel({
            input: ['resources/css/app.css', 'resources/js/app.js'],
            refresh: true,
        }),
    ],
});
```

Points d'entree : `resources/css/app.css` et `resources/js/app.js`. Le mode `refresh: true` active le rechargement automatique lors des modifications des fichiers Blade.

15.2 Configuration Tailwind CSS

Fichier : `tailwind.config.js`

```
export default {
    content: [
        './vendor/laravel/framework/src/Illuminate/Pagination/resources/views/*.blade.php',
        './storage/framework/views/*.php',
        './resources/views/**/*.blade.php',
    ],
    theme: {
        extend: {
            fontFamily: {
                sans: ['Figtree', ...defaultTheme.fontFamily.sans],
            },
        },
    },
    plugins: [forms],
};
```

Le scan des classes Tailwind couvre les fichiers Blade, les vues de pagination Laravel et les vues compilees.

15.3 Configuration PostCSS

Fichier : `postcss.config.js`

```
export default {
    plugins: {
        tailwindcss: {},
        autoprefixer: {},
    },
};
```

Les plugins Tailwind et Autoprefixer sont appliques en post-traitement.

15.4 Feuille de style personnalisee

Le fichier `public/css/style.css` contient les styles specifiques au centre equestre, en complement de Bootstrap et Tailwind. Les couleurs principales (`#bf9b6e` , `#340604` , `#fcf9f2`) y sont definies.

15.5 Commandes de build

Commande	Role
`npm run dev`	Serveur de developpement Vite avec hot reload (port 5173)
`npm run build`	Compilation pour production (fichiers optimises dans `public/build/`)

16. Seeders et factories

16.1 DatabaseSeeder (orchestrateur)

Fichier : `database/seeders/DatabaseSeeder.php`

Ordre d'execution :

1. Creation de 3 utilisateurs (`admin@example.com` , `enseignant@example.com` , `user@example.com`) avec le mot de passe `password`
2. Attribution des roles Bouncer (admin, enseignant)
3. Attribution des permissions `manage-events` et `view-planning` au role enseignant
4. Appel des seeders specialises (dans l'ordre) :
    - `EventTypeSeeder`
    - `EventSeeder`
    - `HorseSeeder`
    - `TarifSeeder`
    - `GallerySeeder`
    - `PressReviewSeeder`
    - `StageSeeder`
5. Attribution aleatoire de types (cours/concours) aux evenements
6. Inscription de l'administrateur a 3 evenements aleatoires

16.2 Seeders specialises

Seeder	Donnees creees
`EventTypeSeeder`	2 types : "cours", "concours"
`EventSeeder`	10 evenements via factory
`HorseSeeder`	10 chevaux + 10 poneys via factory
`TarifSeeder`	Grille tarifaire complete (12 tarifs cheval + 9 tarifs poney)
`GallerySeeder`	3 galeries avec photos : "Concours CSO Internes" (5 photos), "Fete du Club 2024" (8 photos), "Stage de Paques" (1 photo)
`PressReviewSeeder`	3 articles de presse
`StageSeeder`	2 stages avec planning journalier et tarifs : "Stage Perfectionnement Saut" (3 jours, 3 tarifs) et "Decouverte Equitation" (2 jours, 3 tarifs)
`TariffSeeder`	Vide (non utilise)

16.3 Factories

Factory	Champs generes

`UserFactory`	name (Faker), email (unique), phone_number, email_verified_at (now), password ('password' hashe), remember_token
`EventFactory`	name (3 mots), description (paragraphe), start_date (-1 a +2 mois), end_date (+1 a 4h apres debut)
`HorseFactory`	name (prenom), birth_date, photo_path (null), type (cheval ou poney aleatoire)
`GalleryFactory`	title (3 mots), date, cover_image ('img/default.jpg')
`GalleryPhotoFactory`	gallery_id (factory), image_path ('img/photo.jpg'), title (optionnel)
`PressReviewFactory`	title, description (paragraphe), date, link (URL)
`TarifFactory`	section, title, description, price (10-100), promo_text (optionnel), frequency (optionnel)

17. Tests et qualite de code

17.1 Framework de tests

L'application utilise **Pest** (version 4.1), une surcouche de PHPUnit offrant une syntaxe plus expressive.

17.2 Configuration (phpunit.xml)

```xml
<testsuites>
    <testsuite name="Unit">
        <directory>tests/Unit</directory>
    </testsuite>
    <testsuite name="Feature">
        <directory>tests/Feature</directory>
    </testsuite>
</testsuites>
```

Variables d'environnement de test :

Variable	Valeur	Role
`APP_ENV`	testing	Environnement de test
`BCRYPT_ROUNDS`	4	Hashage rapide
`CACHE_STORE`	array	Cache en memoire
`DB_DATABASE`	testing	Base de donnees separee
`MAIL_MAILER`	array	Pas d'envoi reel d'emails
`QUEUE_CONNECTION`	sync	Execution synchrone des jobs
`SESSION_DRIVER`	array	Sessions en memoire

17.3 Commandes de test

```bash
# Execution de tous les tests
composer test
# ou
./vendor/bin/pest

# Tests d'une suite specifique
./vendor/bin/pest tests/Feature
./vendor/bin/pest tests/Unit

# Avec couverture de code
./vendor/bin/pest --coverage
```

17.4 Analyse statique (PHPStan / Larastan)

Fichier : `phpstan.neon`

```yaml
includes:
    - vendor/larastan/larastan/extension.neon
```

```yaml
parameters:
    paths:
        - app/
        - database/
    level: 7
```

Le niveau 7 est utilise (sur une echelle de 0 a 9), offrant une analyse stricte couvrant : types de retour, types de parametres, appels de methodes, proprietes, etc.

```
./vendor/bin/phpstan analyse
```

17.5 Formatage de code

Le formatage du code PHP est assure par **Laravel Pint**, qui applique les regles PSR-12 :

```
./vendor/bin/pint
```

18. Environnement Docker et deploiement

18.1 Architecture des conteneurs

L'application utilise **Laravel Sail** pour la conteneurisation. La configuration est definie dans `compose.yaml`.

```
+-----------------------------------------------------+
|              Reseau Docker : sail (bridge)           |
|                                                     |
|  +-------------------+  +--------------------+      |
|  | laravel.test      |  | mariadb            |      |
|  | PHP 8.4 + Nginx   |  | MariaDB 11         |      |
|  | Ports: 80, 5173   |--| Port: 3306         |      |
|  +-------------------+  +--------------------+      |
|                                                     |
|  +-------------------+  +--------------------+      |
|  | redis             |  | mailpit            |      |
|  | Redis Alpine      |  | SMTP: 1025         |      |
|  | Port: 6379        |  | Dashboard: 8025    |      |
|  +-------------------+  +--------------------+      |
+-----------------------------------------------------+
```

18.2 Service laravel.test

Parametre	Valeur
Image	`sail-8.4/app` (construite depuis `vendor/laravel/sail/runtimes/8.4`)
Ports	`${APP_PORT:-80}:80`, `${VITE_PORT:-5173}:${VITE_PORT:-5173}`
Volume	`.:/var/www/html` (montage du code source)
Depends_on	mariadb, redis, mailpit
Extra hosts	`host.docker.internal:host-gateway` (acces au host)
Xdebug	Configurable via `SAIL_XDEBUG_MODE` (off par defaut)

18.3 Service mariadb

Parametre	Valeur
Image	`mariadb:11`
Port	`${FORWARD_DB_PORT:-3306}:3306`
Volume	`sail-mariadb:/var/lib/mysql` (persistance)
Script d'init	`create-testing-database.sh` (cree automatiquement la base de test)
Healthcheck	`healthcheck.sh --connect --innodb_initialized`

18.4 Service redis

Parametre	Valeur

Image	`redis:alpine`
Port	`${FORWARD_REDIS_PORT:-6379}:6379`
Volume	`sail-redis:/data` (persistance)
Healthcheck	`redis-cli ping`

18.5 Service mailpit

Parametre	Valeur
Image	`axllent/mailpit:latest`
Port SMTP	`${FORWARD_MAILPIT_PORT:-1025}:1025`
Port Dashboard	`${FORWARD_MAILPIT_DASHBOARD_PORT:-8025}:8025`

Mailpit intercepte tous les emails envoyes par l'application et les affiche dans un dashboard web accessible sur le port 8025.

18.6 Volumes persistants

Volume	Montage	Role
`sail-mariadb`	`/var/lib/mysql`	Donnees MariaDB
`sail-redis`	`/data`	Donnees Redis

18.7 Commandes Docker (Sail)

```
# Demarrer les conteneurs
./vendor/bin/sail up -d

# Arreter les conteneurs
./vendor/bin/sail down

# Executer une commande artisan
./vendor/bin/sail artisan migrate

# Acceder au shell du conteneur
./vendor/bin/sail shell

# Voir les logs
./vendor/bin/sail logs

# Executer npm
./vendor/bin/sail npm run build

# Executer les tests
./vendor/bin/sail test
```

19. Configuration de l'environnement

19.1 Variables d'environnement (.env)

Variable	Valeur dev	Description
`APP_NAME`	Laravel	Nom de l'application
`APP_ENV`	local	Environnement (local, staging, production)
`APP_KEY`	(generee)	Cle de chiffrement AES-256-CBC
`APP_DEBUG`	true	Mode debug (false en production)
`APP_URL`	http://localhost	URL de base
`DB_CONNECTION`	mysql	Driver de connexion
`DB_HOST`	mariadb	Hote de la base de donnees
`DB_PORT`	3306	Port de la base de donnees
`DB_DATABASE`	laravel	Nom de la base
`DB_USERNAME`	sail	Utilisateur BDD
`DB_PASSWORD`	password	Mot de passe BDD
`CACHE_STORE`	redis	Driver de cache
`SESSION_DRIVER`	redis	Driver de sessions
`QUEUE_CONNECTION`	redis	Driver de files d'attente
`REDIS_HOST`	redis	Hote Redis
`REDIS_PORT`	6379	Port Redis
`MAIL_MAILER`	smtp	Transport email
`MAIL_HOST`	mailpit	Serveur SMTP
`MAIL_PORT`	1025	Port SMTP
`VITE_PORT`	5173	Port du serveur Vite

19.2 Configuration pour la production

Modifications recommandees pour un deploiement en production :

```
APP_ENV=production
APP_DEBUG=false
APP_URL=https://www.domaine-du-centre.fr

DB_HOST=serveur-bdd-production
DB_DATABASE=nom_base_production
DB_USERNAME=utilisateur_production
DB_PASSWORD=mot_de_passe_securise

MAIL_MAILER=smtp
MAIL_HOST=smtp.fournisseur.com
```

```
MAIL_PORT=587
MAIL_ENCRYPTION=tls
MAIL_USERNAME=email@domaine.fr
MAIL_PASSWORD=mot_de_passe_email

CACHE_STORE=redis
SESSION_DRIVER=redis
```

19.3 Optimisation pour la production

```
php artisan config:cache    # Cache de la configuration
php artisan route:cache     # Cache des routes
php artisan view:cache      # Cache des vues compilees
php artisan optimize        # Optimisation globale
npm run build               # Compilation des assets
```

20. Maintenance et commandes utiles

20.1 Commandes Artisan principales

Commande	Description
`php artisan serve`	Demarrer le serveur de developpement
`php artisan migrate`	Executer les migrations
`php artisan migrate:rollback`	Annuler la derniere migration
`php artisan migrate:status`	Voir l'etat des migrations
`php artisan db:seed`	Peupler la base de donnees
`php artisan db:table users`	Afficher le schema d'une table
`php artisan route:list`	Lister toutes les routes
`php artisan tinker`	Console interactive PHP
`php artisan cache:clear`	Vider le cache applicatif
`php artisan config:clear`	Vider le cache de configuration
`php artisan route:clear`	Vider le cache des routes
`php artisan view:clear`	Vider le cache des vues

20.2 Commandes de creation

Commande	Description
`php artisan make:controller NomController`	Creer un controleur
`php artisan make:model NomModele -m`	Creer un modele avec migration
`php artisan make:migration create_nom_table`	Creer une migration
`php artisan make:seeder NomSeeder`	Creer un seeder
`php artisan make:factory NomFactory`	Creer une factory
`php artisan make:request NomRequest`	Creer un form request
`php artisan make:middleware NomMiddleware`	Creer un middleware

20.3 Logs et debugging

```
# Consulter les logs en temps reel
tail -f storage/logs/laravel.log

# Ou via Laravel Pail
php artisan pail

# Dashboard Mailpit (emails)
# http://localhost:8025
```

```
# Route de sante
# GET /up
```

20.4 Scripts Composer

Script	Commande
`composer setup`	Installation complete (dependances, cle, migrations, build)
`composer dev`	Demarrage du serveur de dev + queue + Vite en parallele
`composer test`	Nettoyage config + execution des tests Pest

21. Axes d'evolution

21.1 Fonctionnalites identifiees

Fonctionnalite	Description	Complexite
Interface d'administration	CRUD complet pour events, users, horses, tarifs via un back-office	Elevee
Multi-tenant (Bouncer scoping)	Activation du `ScopeBouncer` pour gerer plusieurs centres	Moyenne
Notifications automatiques	Rappels par email (cours, stages, delais d'inscription)	Moyenne
Paiement en ligne	Integration Stripe ou PayPal pour les inscriptions et recharges de carte	Elevee
API REST	Exposition d'une API pour une application mobile	Elevee
Export de donnees	Generation de CSV/PDF des inscriptions et statistiques	Faible
Gestion des chevaux enrichie	Planning des chevaux (table `horse_schedules` deja creee)	Moyenne
Systeme de notation	Evaluation des cours par les cavaliers	Faible

21.2 Ameliorations techniques identifiees

Amelioration	Description
Augmenter le niveau PHPStan	Passer du niveau 7 au niveau 8 ou 9 pour une analyse plus stricte
Couverture de tests	Ajouter des tests fonctionnels pour les controleurs principaux (Planning, Stages)
Internationalisation	Ajouter le support multi-langue via les fichiers de traduction Laravel
Mise en cache	Cacher les requetes frequentes (tarifs, cavalerie, galeries)
Observateurs	Utiliser des observers Laravel pour la logique de creation d'adherent
Politique d'autorisation	Remplacer les verifications manuelles par des Policy Laravel
Validation enrichie	Creer des FormRequest dedies pour l'inscription aux cours

---

Document redige dans le cadre du BTS SIO option SLAM -- Mars 2026

Application developpee avec le framework Laravel 12 -- PHP 8.4

Centre Equestre de Pontchateau -- Loire-Atlantique, France